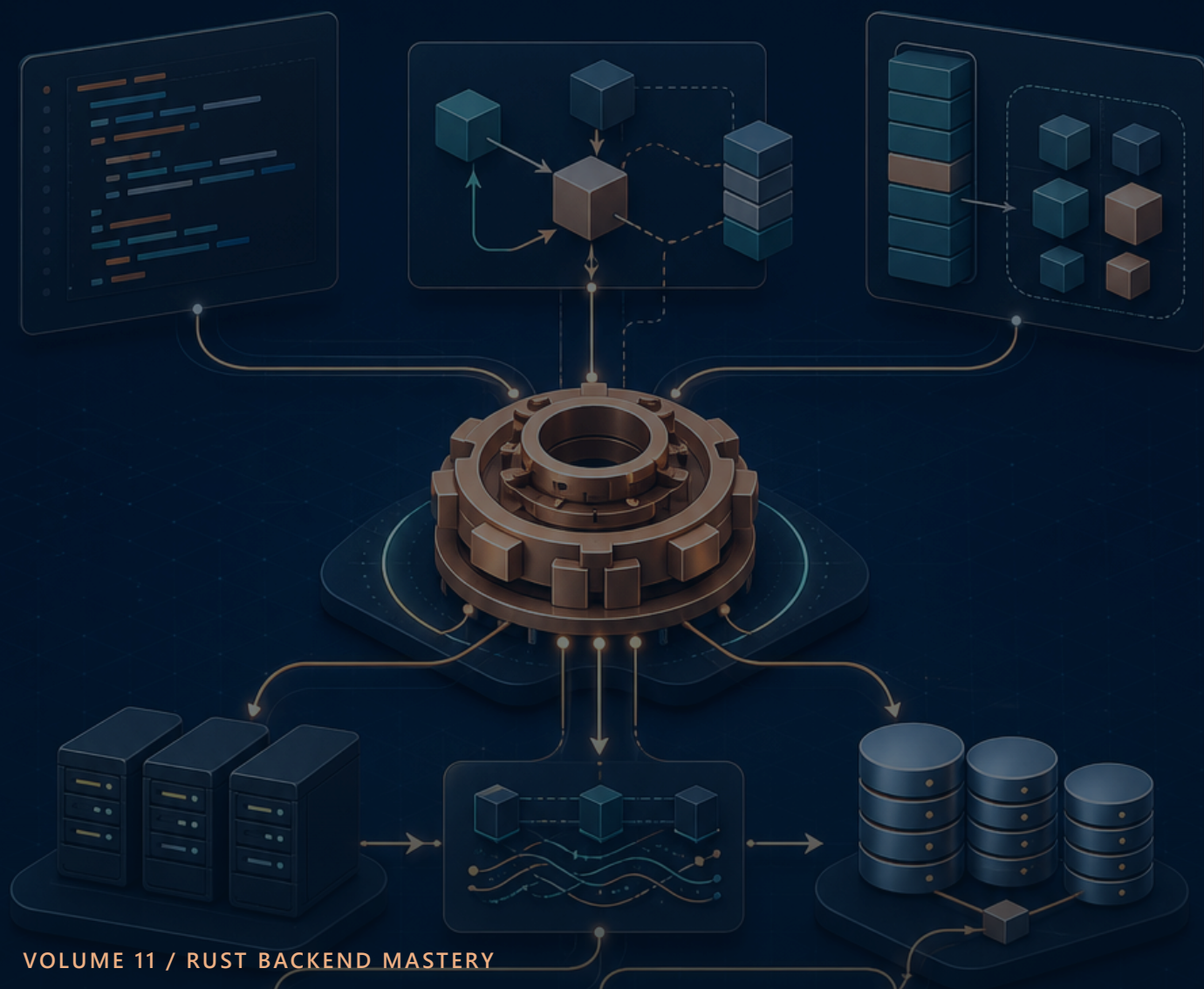




VOLUME 11 / RUST BACKEND MASTERY

品質保証 — テスト・セキュリティ・可観測性・性能

単体からE2E、security、SLO、benchmark、load、CIまで、何をどの層で証明するかを実装する

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

テスト件数ではなく、守る不変条件、失敗artifact、利用者影響、release判断までを品質の成果にします。

到達目標

- ✓ test pyramidを責務で使い分けられる
- ✓ security regressionを具体payloadで書ける
- ✓ SLOとobservabilityを設計できる
- ✓ 性能・capacity・CIを測定と証跡で運用できる

学習地図

01 unit test・境界値・table-driven

純粋なdomain関数を高速に検証します。代表値だけでなく空、最小、最大、最大+1、日本語、空白、overflowを表形式で並べ、失敗理由も比較します。

02 Repository integration test

実際のlibSQL schemaとSQLを使い、migration、bind、row mapping、constraint、transactionを検証します。testごとに隔離DBを作り、実行順へ依存させません。

03 HTTP handler・status・header test

routeをprocess内で呼び、status、Location、Content-Type、body、cookie属性を検証します。成功本文だけでなく400、redirect、404、503を固定します。

04 property test・fuzz・parser

個別caseでは見落とす広い入力空間をpropertyで検証します。parserは任意文字列でpanicせず、成功値が不変条件を満たし、serialize/parse roundtripが成立することを確認します。

05 E2E・browser・accessibility

実browserでregisterからlogoutまで操作し、visible result、URL、focus、Network、responsiveを確認します。E2Eは少数の重要journeyに絞り、下層testと役割を分けます。

06 security regression test

脅威モデルを実行可能な負のtestへ変換します。SQL injection、XSS、CSRF、IDOR、session fixation、cookie属性、rate limit、secret exposureをrelease条件にします。

07 tracing・metrics・health・SLO

log、metric、trace、healthは目的が違います。SLOをavailabilityとlatencyで決め、request rate、error rate、duration、saturationを測り、利用者影響に結びます。

08 benchmark・profile・allocation

性能改善は計測前に行いません。release buildで代表入力をbenchmarkし、wall time、CPU、allocation、query数を測り、profileでhot spotを特定します。

09 load test・backpressure・capacity

負荷試験は平均だけでなくp95/p99、error、DB latency、queue、CPU、memoryを一定rateで観測します。過負荷で無制限待機せず、Semaphore・queue・timeoutで劣化を制御します。

10 CI・dependency・supply chain・incident

CIはfmt、clippy、test、security test、build、asset bundle、container smokeを順に実行します。依存監査、license、secret scan、SBOM、provenanceを追加し、incident時のrollbackと証跡を準備します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。



毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```

cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
  
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

unit test・境界値・table-driven

UNIT 01・CONCEPT

unit test・境界値・table-driven

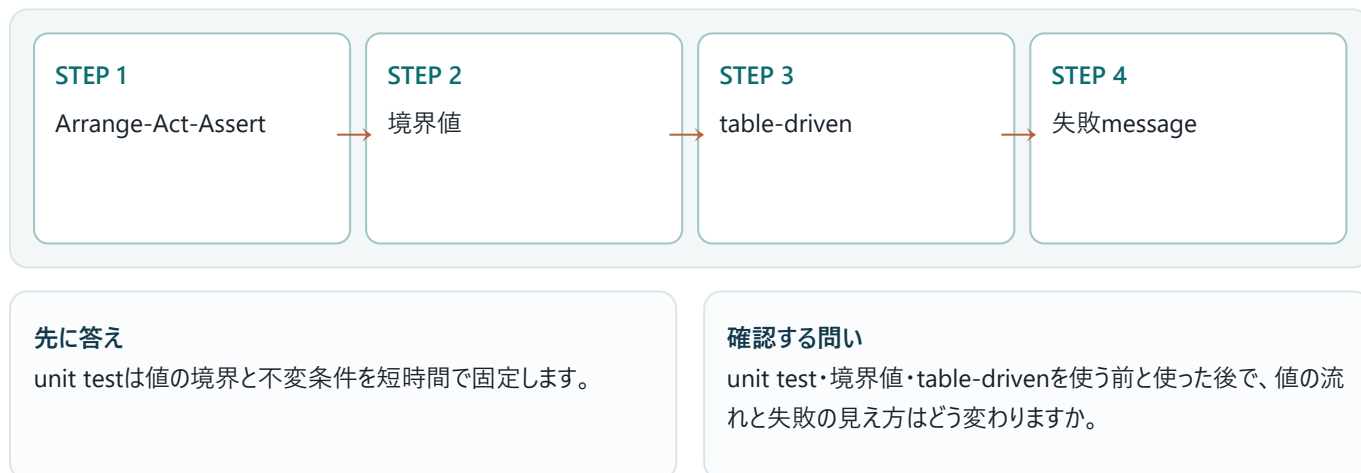
純粋なdomain関数を高速に検証します。代表値だけでなく空、最小、最大、最大+1、日本語、空白、overflowを表形式で並べ、失敗理由も比較します。

この単元の到達点

- Arrange-Act-Assert
- 境界値
- table-driven
- 失敗message

なぜバックエンドで必要か

frameworkなしで不変条件を固定すると、web・DB変更後もdomain契約を速く確認できます。



UNIT 01・MENTAL MODEL

unit test・境界値・table-drivenを図でつかむ



純粋なdomain関数を高速に検証します。代表値だけでなく空、最小、最大、最大+1、日本語、空白、overflowを表形式で並べ、失敗理由も比較します。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Arrange-Act-Assert	Arrange-Act-Assert — 入力と前提を確認し、境界を曖昧にしない。
2	境界値	境界値 — 値がどの順で変換されるかを追跡する。
3	table-driven	table-driven — 失敗を再現し、型またはログから原因を特定する。
4	失敗message	失敗message — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

unit test・境界値・table-drivenとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

unit test・境界値・table-drivenだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[test]fn title_cases(){for(raw,ok)in[("Rust",true),(" ",false),(&"あ".repeat(80),true),(&"あ".repeat(81),false)]{assert_eq!(Title::parse(raw).is_ok(),ok,"raw length={}",raw.chars().count());}}
```

● ● ● 実行コマンド

```
cargo test domain -- --nocapture
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[test]fn title_cases(){for(raw,ok)in[("Rust",true),(" ",false),(&"あ".repe</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

コード／場所	役割と理由
unit test・境界値・table-driven	入力、処理、出力を分離して読みます。
unit test・境界値・table-driven	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がArrange-Act-Assertの条件を満たすか確認する。
- ✓ 境界値を実行し、途中の型と状態を追う。
- ✓ table-drivenで失敗を成功経路から分離する。
- ✓ 失敗messageにより、出力と副作用を検証する。

型を声に出す unit test・境界値・table-drivenに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
test title_cases ... ok
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
assert!(matches!(NoteId::new(0), Err(IdError::NonPositive))); assert!(matches!(Title::parse(""), Err(TitleError::Empty))); assert!(matches!(Title::parse(&"x".repeat(81)), Err(TitleError::TooLong{..})));
```


変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

frameworkなしで不変条件を固定すると、web・DB変更後もdomain契約を速く確認できます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Arrange-Act-Assertに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	境界値の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	table-drivenまたは失敗messageの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

unit test・境界値・table-drivenを再現して切り分ける

unit test・境界値・table-drivenの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Arrange-Act-Assert
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 境界値
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: table-driven
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 失敗message

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Arrange-Act-Assert	Arrange-Act-Assert — 入力と前提を確認し、境界を曖昧にしない。
2	境界値	境界値 — 値がどの順で変換されるかを追跡する。
3	table-driven	table-driven — 失敗を再現し、型またはログから原因を特定する。
4	失敗message	失敗message — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 unit testは値の境界と不変条件を短時間で固定します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

Email、NoteTitle、NoteIdの境界case表を作り、各失敗variantまでassertしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
assert!(matches!(NoteId::new(0), Err(IdError::NonPositive))); assert!(matches!(Title::parse("
"), Err(TitleError::Empty))); assert!(matches!(Title::parse(&"x".repeat(81)), Err(TitleError::TooLong{..})));
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ unit testは値の境界と不変条件を短時間で固定します。
- ✓ unit test・境界値・table-drivenとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

Repository integration test

UNIT 02 · CONCEPT

Repository integration test

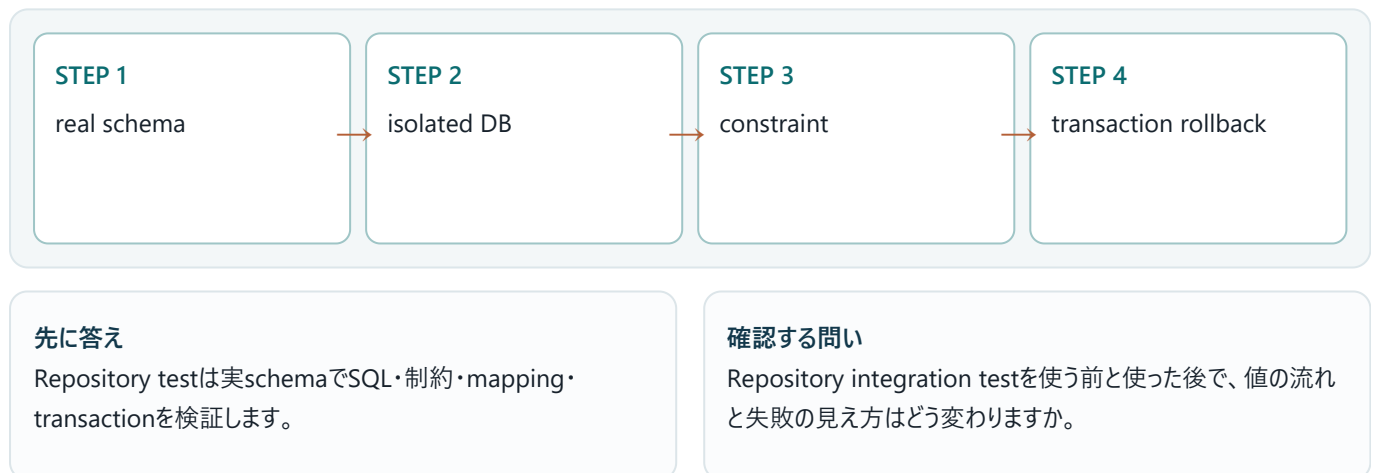
実際のlibSQL schemaとSQLを使い、migration、bind、row mapping、constraint、transactionを検証します。testごとに隔離DBを作り、実行順へ依存させません。

この単元の到達点

- real schema
- isolated DB
- constraint
- transaction rollback

なぜバックエンドで必要か

mockだけではSQL type、列順、migration差を検出できません。remote Tursoは別の少数contract testで補います。



UNIT 02 · MENTAL MODEL

Repository integration testを図でつかむ



実際のlibSQL schemaとSQLを使い、migration、bind、row mapping、constraint、transactionを検証します。testごとに隔離DBを作り、実行順へ依存させません。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	real schema	real schema — 入力と前提を確認し、境界を曖昧にしない。
2	isolated DB	isolated DB — 値がどの順で変換されるかを追跡する。
3	constraint	constraint — 失敗を再現し、型またはログから原因を特定する。
4	transaction rollback	transaction rollback — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Repository integration testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE

まず動く最小コード

Repository integration testだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[tokio::test]async fn note_is_scoped_to_owner(){let db=test_db().await;let a=seed_user(&db).await;let b=seed_user(&db).await;let n=insert_note(&db,b.id,"secret").await;assert!(repo(&db).find_owned(a.id,n.id).await.unwrap().is_none());}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[tokio::test]async fn note_is_scoped_to_owner(){let db=test_db().await;le</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
Repository integration test	入力、処理、出力を分離して読みます。
Repository integration test	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がある schema の条件を満たすか確認する。
- ✓ isolated DB を実行し、途中の型と状態を追う。
- ✓ constraint で失敗を成功経路から分離する。
- ✓ transaction rollback により、出力と副作用を検証する。

型を声に出す Repository integration test に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

test note_is_scoped_to_owner ... ok

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
assert!(insert_user(&db,"same@example.com").await.is_ok());assert!(matches!(insert_user(&db,"same@example.com").await,Err(RepoError::Conflict)));
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

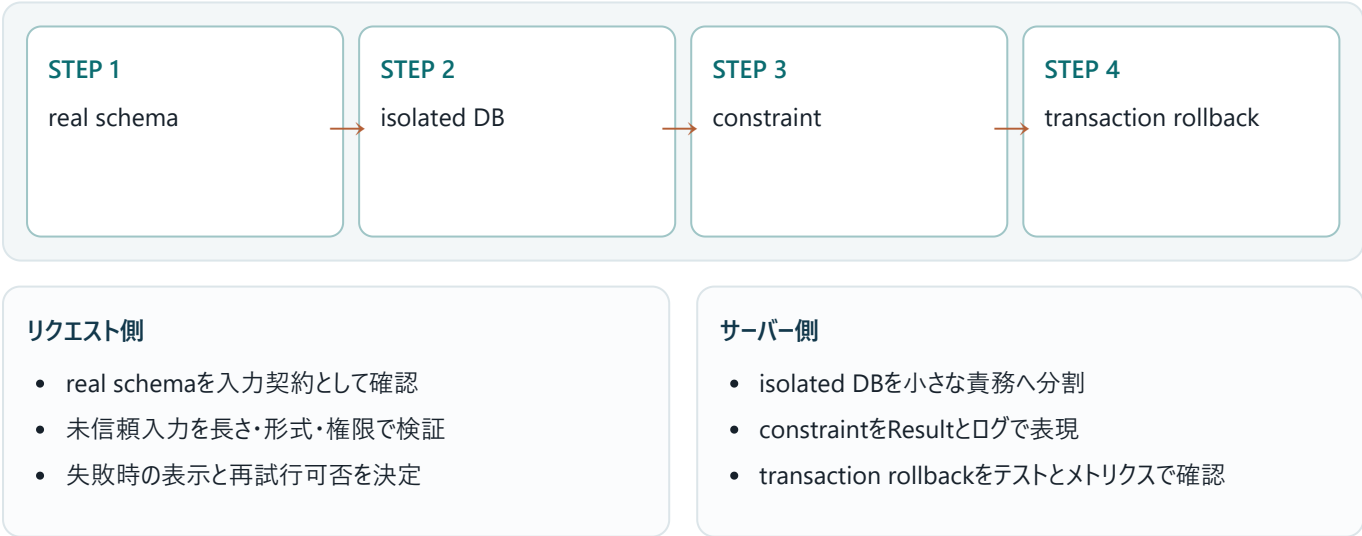
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

mockだけではSQL typo、列順、migration差を検出できません。remote Tursoは別の少数contract testで補います。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	real schemaに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	isolated DBの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	constraintまたはtransaction rollbackの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

Repository integration testを再現して切り分ける

Repository integration testの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: real schema
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: isolated DB
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: constraint
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: transaction rollback

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	real schema	real schema — 入力と前提を確認し、境界を曖昧にしない。
2	isolated DB	isolated DB — 値がどの順で変換されるかを追跡する。
3	constraint	constraint — 失敗を再現し、型またはログから原因を特定する。
4	transaction rollback	transaction rollback — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 Repository testは実schemaでSQL・制約・mapping・transactionを検証します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

UNIQUE email、FK cascade、title CHECK、rollbackを一件ずつ壊して期待errorをassertしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
assert!(insert_user(&db, "same@example.com").await.is_ok()); assert!(matches!(insert_user(&db, "same@example.com").await, Err(RepoError::Conflict)));
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ Repository testは実schemaでSQL・制約・mapping・transactionを検証します。
- ✓ Repository integration testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

HTTP handler・status・header test

UNIT 03 ・ CONCEPT

HTTP handler・status・header test

routeをprocess内で呼び、status、Location、Content-Type、body、cookie属性を検証します。成功本文だけでなく400、redirect、404、503を固定します。

この単元の到達点

- request builder
- status
- headers
- body/cookie

なぜバックエンドで必要か

HTTP testはdomainとDBを統合し、利用者とproxyが見る外部契約を固定します。internal error文字列がbodyへ漏れないことも確認します。



UNIT 03 ・ MENTAL MODEL

HTTP handler・status・header testを図でつかむ



routeをprocess内で呼び、status、Location、Content-Type、body、cookie属性を検証します。成功本文だけでなく400、redirect、404、503を固定します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認しま

す。

順序	観察対象	初心者が確認すること
1	request builder	request builder — 入力と前提を確認し、境界を曖昧にしない。
2	status	status — 値がどの順で変換されるかを追跡する。
3	headers	headers — 失敗を再現し、型またはログから原因を特定する。
4	body/cookie	body/cookie — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

HTTP handler・status・header testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03・MINIMUM CODE

まず動く最小コード

HTTP handler・status・header testだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
let response=app.request(Request::post("/notes").form(&[("title","Rust")]).session(user)).await;assert_eq!(response.status(),303);assert_eq!(response.header("location"),Some("/notes"));
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
let response=app.request(Request::post("/notes").form(&[("title","Rust")]))	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
HTTP handler・status・header test	入力、処理、出力を分離して読みます。
HTTP handler・status・header test	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がrequest builderの条件を満たすか確認する。
- ✓ statusを実行し、途中の型と状態を追う。
- ✓ headersで失敗を成功経路から分離する。
- ✓ body/cookieにより、出力と副作用を検証する。

型を声に出す HTTP handler・status・header testに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。



見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
assert_eq!(post_without_session().await.status(), 302); assert_eq!(
    post_empty_title().await.status(), 400); assert_eq!(delete_other_user().await.status(), 404); assert_eq!(
    simulate_db_timeout().await.status(), 503);
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 · BACKEND APPLICATION

バックエンドのどこで使うか

HTTP testはdomainとDBを統合し、利用者とproxyが見る外部契約を固定します。internal error文字列がbodyへ漏れないことも確認します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	request builderに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	statusの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	headersまたはbody/cookieの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

HTTP handler・status・header testを再現して切り分ける

HTTP handler・status・header testの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: request builder
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: status
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: headers
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: body/cookie

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	request builder	request builder — 入力と前提を確認し、境界を曖昧にしない。
2	status	status — 値がどの順で変換されるかを追跡する。
3	headers	headers — 失敗を再現し、型またはログから原因を特定する。
4	body/cookie	body/cookie — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 外部契約はstatus・header・body・cookieを組み合わせて検証します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

未認証、invalid content-type、空title、他人ID、DB timeoutのstatusとbodyをtestしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
assert_eq!(post_without_session().await.status(),302);assert_eq!(
    post_empty_title().await.status(),400);assert_eq!(delete_other_user().await.status(),404);assert_eq!(
    simulate_db_timeout().await.status(),503);
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 外部契約はstatus・header・body・cookieを組み合わせて検証します。
- ✓ HTTP handler・status・header testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

property test・fuzz・parser

UNIT 04 · CONCEPT

property test・fuzz・parser

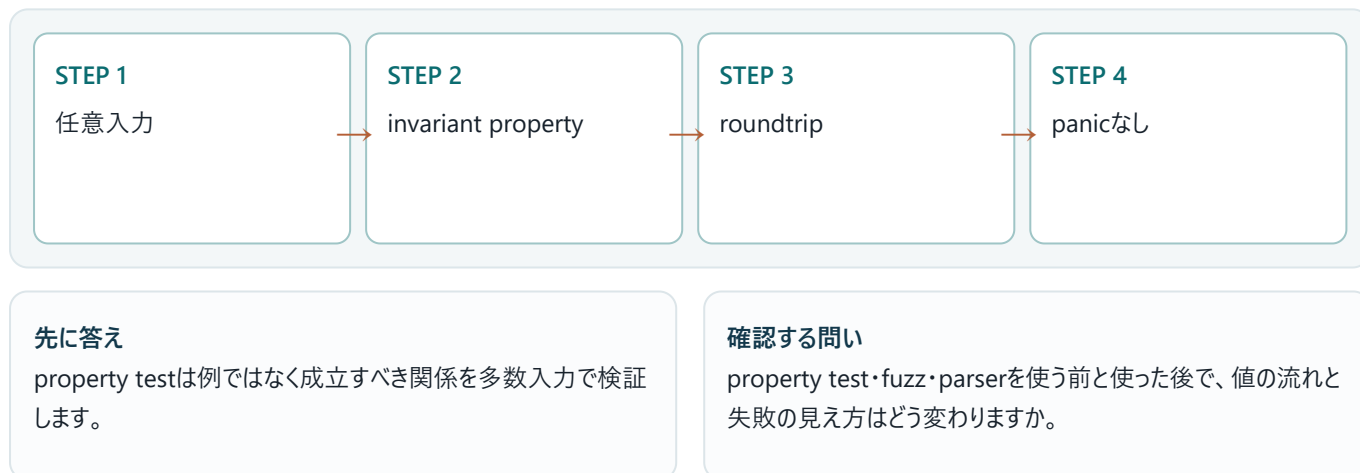
個別caseでは見落とす広い入力空間をpropertyで検証します。parserは任意文字列でpanicせず、成功値が不変条件を満たし、serialize/parse roundtripが成立することを確認します。

この単元の到達点

- 任意入力
- invariant property
- roundtrip
- panicなし

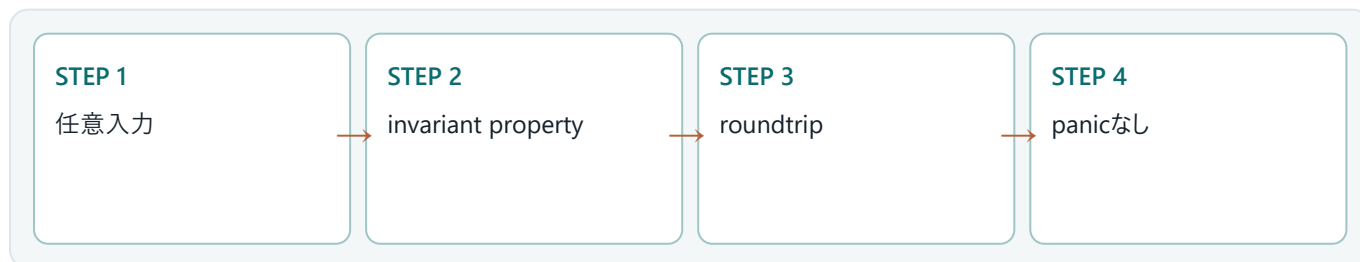
なぜバックエンドで必要か

path/query/cursor/token parserは攻撃者が任意byteを送れます。fuzz targetでpanic、巨大割当、極端な遅さを探します。



UNIT 04 · MENTAL MODEL

property test・fuzz・parserを図でつかむ



個別caseでは見落とす広い入力空間をpropertyで検証します。parserは任意文字列でpanicせず、成功値が不変条件を満たし、serialize/parse roundtripが成立することを確認します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	任意入力	任意入力 — 入力と前提を確認し、境界を曖昧にしない。
2	invariant property	invariant property — 値がどの順で変換されるかを追跡する。
3	roundtrip	roundtrip — 失敗を再現し、型またはログから原因を特定する。
4	panicなし	panicなし — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

property test・fuzz・parserとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE

まず動く最小コード

property test・fuzz・parserだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
proptest!{#[test]fn normalized_title_never_exceeds_80(s in ".*"){if let Ok(t)=Title::parse(&s){prop_assert!((1..=80).contains(&t.as_str().chars().count()));}}}
```

● ● ● 実行コマンド

```
cargo test property
# 任意: cargo fuzz run cursor
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>proptest!{#[test]fn normalized_title_never_exceeds_80(s in ".*"){if let Ok</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>property test・fuzz・parser</code>	入力、処理、出力を分離して読みます。
<code>property test・fuzz・parser</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力が任意入力の条件を満たすか確認する。
- ✓ `invariant property`を実行し、途中の型と状態を追う。
- ✓ `roundtrip`で失敗を成功経路から分離する。
- ✓ `panic`なしにより、出力と副作用を検証する。

型を声に出す `property test・fuzz・parser`に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
proptest: 256 cases ... ok
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
proptest!{#[test]fn cursor_roundtrip(ts in any::<i64>(),id in 1i64..i64::MAX){let
c=Cursor{ts,id};prop_assert_eq!(decode(&encode(&c)).unwrap(),c);}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

path/query/cursor/token parserは攻撃者が任意byteを送れます。fuzz targetでpanic、巨大割当、極端な遅さを探します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	任意入力に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	invariant propertyの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	roundtripまたはpanicなしの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

property test・fuzz・parserを再現して切り分ける

property test・fuzz・parserの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 任意入力
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: invariant property
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: roundtrip
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: panicなし

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	任意入力	任意入力 — 入力と前提を確認し、境界を曖昧にしない。
2	invariant property	invariant property — 値がどの順で変換されるかを追跡する。
3	roundtrip	roundtrip — 失敗を再現し、型またはログから原因を特定する。
4	panicなし	panicなし — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 property testは例ではなく成立すべき関係を多数入力で検証します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

cursor encode/decodeのroundtripと、任意文字列decodeがpanicしないpropertyを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
proptest!{#[test]fn cursor_roundtrip(ts in any::<i64>(),id in 1i64..i64::MAX){let c=Cursor{ts,id};prop_assert_eq!(decode(&encode(&c)).unwrap(),c);}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ property testは例ではなく成立すべき関係を多数入力で検証します。
- ✓ property test・fuzz・parserとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

E2E・browser・accessibility

UNIT 05 ・ CONCEPT

E2E・browser・accessibility

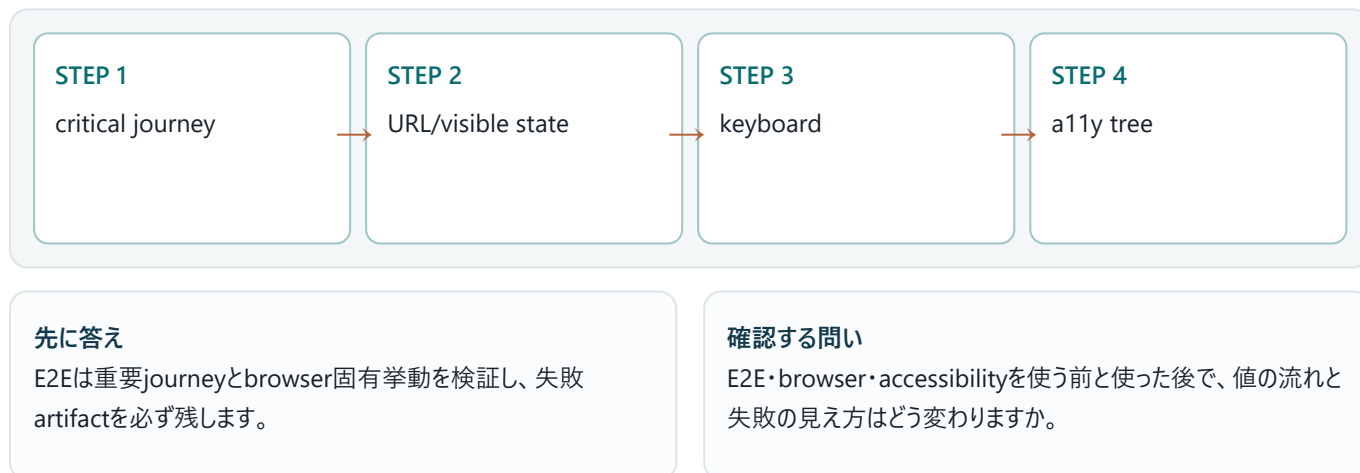
実browserでregisterからlogoutまで操作し、visible result、URL、focus、Network、responsiveを確認します。E2Eは少数の重要journeyに絞り、下層testと役割を分けます。

この単元の到達点

- critical journey
- URL/visible state
- keyboard
- a11y tree

なぜバックエンドで必要か

flakyなsleepではなく要素やnetworkの条件を待ち、失敗時にscreenshot、HTML、console、traceを保存します。



UNIT 05 ・ MENTAL MODEL

E2E・browser・accessibilityを図でつかむ



実browserでregisterからlogoutまで操作し、visible result、URL、focus、Network、responsiveを確認します。E2Eは少数の重要journeyに絞り、下層testと役割を分けます。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	critical journey	critical journey — 入力と前提を確認し、境界を曖昧にしない。
2	URL/visible state	URL/visible state — 値がどの順で変換されるかを追跡する。
3	keyboard	keyboard — 失敗を再現し、型またはログから原因を特定する。
4	a11y tree	a11y tree — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

E2E・browser・accessibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05・MINIMUM CODE

まず動く最小コード

E2E・browser・accessibilityだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
register -> login -> create -> edit -> toggle -> search -> delete -> logout
assert URL and visible heading after each submit
Tab only: reach every control
axe/a11y: no unlabeled field
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
register -> login -> create -> edit -> toggle -> search -> delete -> logou	この行が受け取る値、作る値、副作用の有無を確認します。
assert URL and visible heading after each submit	この行が受け取る値、作る値、副作用の有無を確認します。
Tab only: reach every control	この行が受け取る値、作る値、副作用の有無を確認します。
axe/a11y: no unlabeled field	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力critical journeyの条件を満たすか確認する。
- ✓ URL/visible stateを実行し、途中の型と状態を追う。
- ✓ keyboardで失敗を成功経路から分離する。
- ✓ a11y treeにより、出力と副作用を検証する。

型を声に出す E2E・browser・accessibilityに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

E2E・browser・accessibility

実際の画面遷移、Network、focus order、mobile layoutを一つのtraceで確認します。

観察ポイント

- critical journey
- URL/visible state
- keyboard

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
context_a.login(user_a);context_b.login(user_b);let id=context_b.create_note();context_a.goto(format!("/notes/{id}"));assert_visible("見つかりません");
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05・BACKEND APPLICATION

バックエンドのどこで使うか

flakyなsleepではなく要素やnetworkの条件を待ち、失敗時にscreenshot、HTML、console、traceを保存します。



リクエスト側

- critical journeyを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- URL/visible stateを小さな責務へ分割
- keyboardをResultとログで表現
- a11y treeをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	critical journeyに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	URL/visible stateの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	keyboardまたはa11y treeの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

E2E・browser・accessibilityを再現して切り分ける

E2E・browser・accessibilityの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: critical journey
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: URL/visible state
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: keyboard
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: a11y tree

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	critical journey	critical journey — 入力と前提を確認し、境界を曖昧にしない。
2	URL/visible state	URL/visible state — 値がどの順で変換されるかを追跡する。
3	keyboard	keyboard — 失敗を再現し、型またはログから原因を特定する。
4	a11y tree	a11y tree — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 E2Eは重要journeyとbrowser固有挙動を検証し、失敗artifactを必ず残します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

二人のbrowser contextで同時loginし、他人note URLへの直接移動が404になるE2Eを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
context_a.login(user_a);context_b.login(user_b);let id=context_b.create_note();context_a.goto(format!("/notes/{id}"));assert_visible("見つかりません");
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ E2Eは重要journeyとbrowser固有挙動を検証し、失敗artifactを必ず残します。
- ✓ E2E・browser・accessibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

security regression test

UNIT 06 · CONCEPT

security regression test

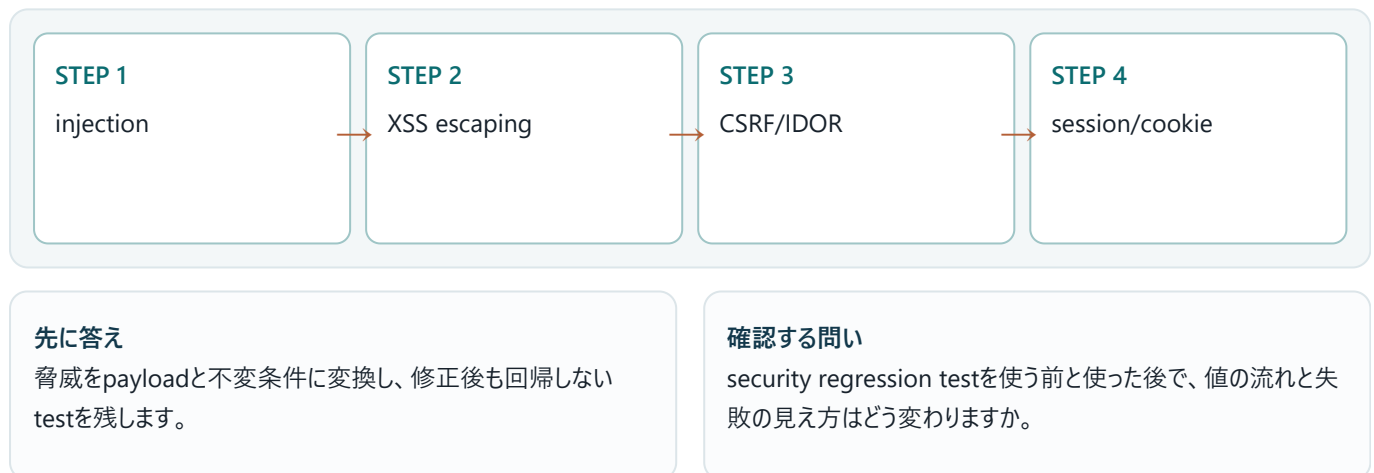
脅威モデルを実行可能な負のtestへ変換します。SQL injection、XSS、CSRF、IDOR、session fixation、cookie属性、rate limit、secret exposureをrelease条件にします。

この単元の到達点

- injection
- XSS escaping
- CSRF/IDOR
- session/cookie

なぜバックエンドで必要か

security testは『攻撃できないはず』ではなく、具体的payload、期待status、DB不変、log非漏えいをassertします。



UNIT 06 · MENTAL MODEL

security regression testを図でつかむ



脅威モデルを実行可能な負のtestへ変換します。SQL injection、XSS、CSRF、IDOR、session fixation、cookie属性、rate limit、secret exposureをrelease条件にします。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	injection	injection — 入力と前提を確認し、境界を曖昧にしない。
2	XSS escaping	XSS escaping — 値がどの順で変換されるかを追跡する。
3	CSRF/IDOR	CSRF/IDOR — 失敗を再現し、型またはログから原因を特定する。
4	session/cookie	session/cookie — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

security regression testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06・MINIMUM CODE

まず動く最小コード

security regression testだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[tokio::test]async fn title_is_escaped(){let body=create_and_render("<script>window.pwned=1</script>").await;assert!(body.contains("&lt;script&gt;"));assert!(!body.contains("<script>window.pwned"));}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[tokio::test]async fn title_is_escaped(){let body=create_and_render("<scr</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
security regression test	入力、処理、出力を分離して読みます。
security regression test	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がinjectionの条件を満たすか確認する。
- ✓ XSS escapingを実行し、途中の型と状態を追う。
- ✓ CSRF/IDORで失敗を成功経路から分離する。
- ✓ session/cookieにより、出力と副作用を検証する。

型を声に出す security regression testに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

malicious title displayed as text
script not executed

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```

CSRF evil Origin→403/no row
IDOR other note→404/unchanged
SQLi title=' OR 1=1--->one literal row only
revoked cookie→redirect/no authenticated action

```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

security testは『攻撃できないはず』ではなく、具体的payload、期待status、DB不変、log非漏えいをassertします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	injectionに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	XSS escapingの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	CSRF/IDORまたはsession/cookieの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

security regression testを再現して切り分ける

security regression testの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: injection
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: XSS escaping
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: CSRF/IDOR
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: session/cookie

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	injection	injection — 入力と前提を確認し、境界を曖昧にしない。
2	XSS escaping	XSS escaping — 値がどの順で変換されるかを追跡する。
3	CSRF/IDOR	CSRF/IDOR — 失敗を再現し、型またはログから原因を特定する。
4	session/cookie	session/cookie — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 脅威をpayloadと不変条件に変換し、修正後も回帰しないtestを残します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

CSRF、IDOR、SQLi、session logout後再利用の4 testをattack→expect→DB invariantで設計してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
CSRF evil Origin→403/no row
IDOR other note→404/unchanged
SQLi title=' OR 1=1-->one literal row only
revoked cookie→redirect/no authenticated action
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 脅威をpayloadと不変条件に変換し、修正後も回帰しないtestを残します。
- ✓ security regression testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

tracing・metrics・health・SLO

UNIT 07・CONCEPT

tracing・metrics・health・SLO

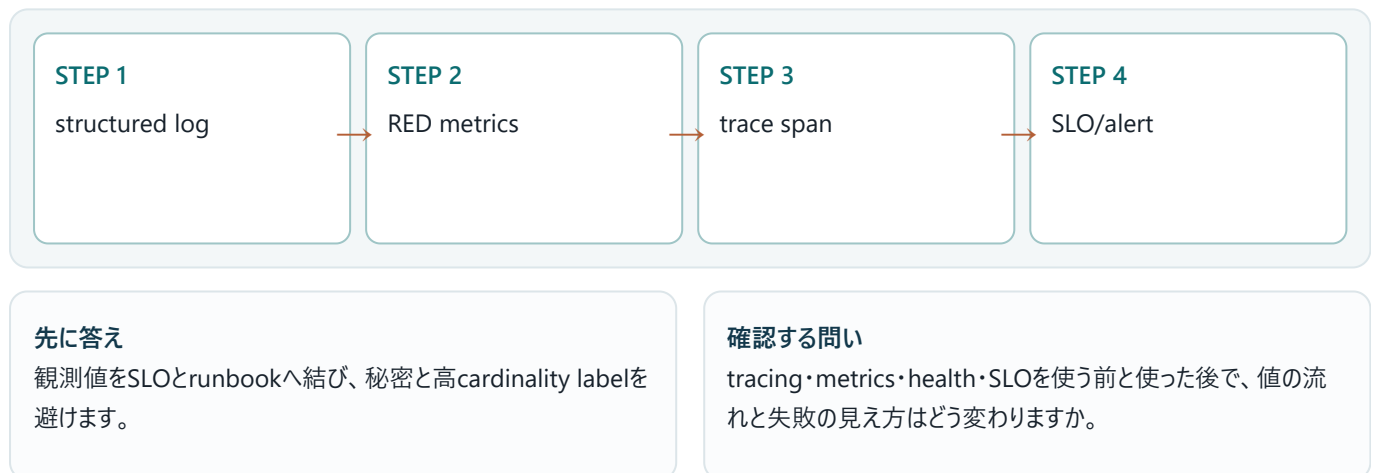
log、metric、trace、healthは目的が違います。SLOをavailabilityとlatencyで決め、request rate、error rate、duration、saturationを測り、利用者影響に結びます。

この単元の到達点

- structured log
- RED metrics
- trace span
- SLO/alert

なぜバックエンドで必要か

route labelはtemplateを使いIDを含めずcardinality爆発を防ぎます。alertはCPUだけでなくerror budgetと利用者症状へ寄せます。



UNIT 07・MENTAL MODEL

tracing・metrics・health・SLOを図でつかむ



log、metric、trace、healthは目的が違います。SLOをavailabilityとlatencyで決め、request rate、error rate、duration、saturationを測り、利用者影響に結びます。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	structured log	structured log — 入力と前提を確認し、境界を曖昧にしない。
2	RED metrics	RED metrics — 値がどの順で変換されるかを追跡する。
3	trace span	trace span — 失敗を再現し、型またはログから原因を特定する。
4	SLO/alert	SLO/alert — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

tracing・metrics・health・SLOとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07・MINIMUM CODE
まず動く最小コード

tracing・metrics・health・SLOだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
info_span!("http.request", request_id, method, path);
histogram!("http.server.duration", elapsed, "route"=>route);
counter!("http.server.requests", 1, "status"=>status_class);
gauge!("job.queue.depth", depth);
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07・LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>info_span!</code> <code>("http.request",request_id,method,path);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>histogram!</code> <code>("http.server.duration",elapsed,"route"=>route);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>counter!</code> <code>("http.server.requests",1,"status"=>status_class);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>gauge!("job.queue.depth",depth);</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がstructured logの条件を満たすか確認する。
- ✓ RED metricsを実行し、途中の型と状態を追う。
- ✓ trace spanで失敗を成功経路から分離する。
- ✓ SLO/alertにより、出力と副作用を検証する。

型を声に出す tracing・metrics・health・SLOに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

dashboard: request/s, 5xx%, p50/p95/p99, DB latency, queue depth

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
Availability: good responses/eligible >=99.9% 30d; alert burn rate
Latency: p95 GET /notes <300ms; alert 15m+1h
Jobs: 99% mail <5m; alert oldest_age>10m
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

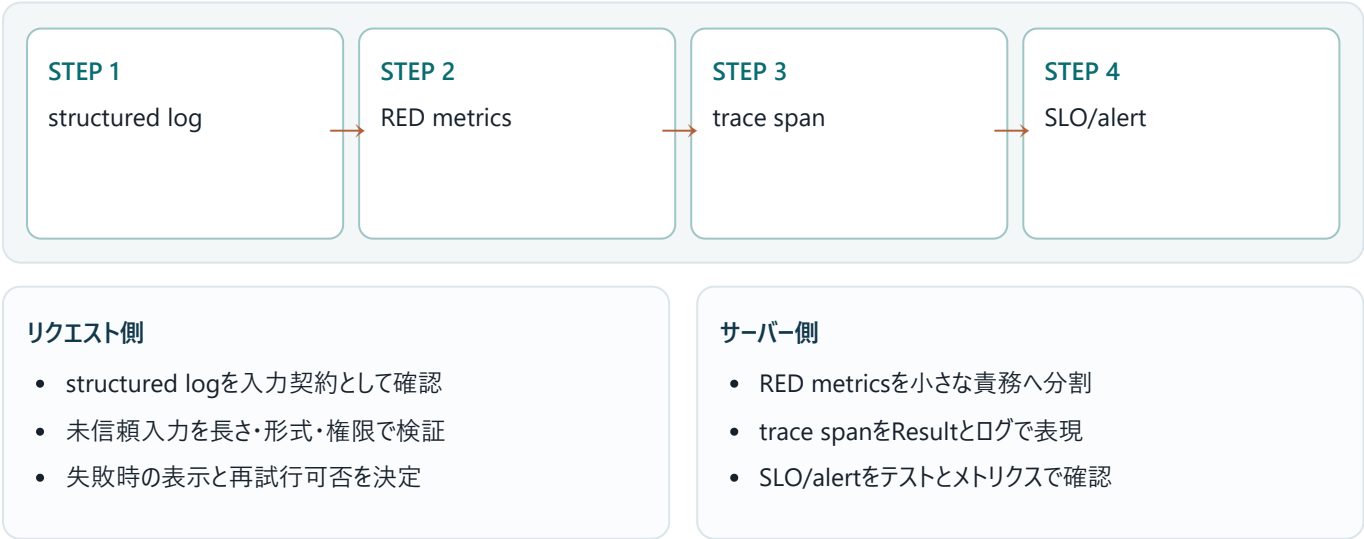
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

route labelはtemplateを使いIDを含めずcardinality爆発を防ぎます。alertはCPUだけでなくerror budgetと利用者症状へ寄せます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	structured logに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	RED metricsの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	trace spanまたはSLO/alertの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

tracing・metrics・health・SLOを再現して切り分ける

tracing・metrics・health・SLOの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: structured log
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: RED metrics
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: trace span
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: SLO/alert

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	structured log	structured log — 入力と前提を確認し、境界を曖昧にしない。
2	RED metrics	RED metrics — 値がどの順で変換されるかを追跡する。
3	trace span	trace span — 失敗を再現し、型またはログから原因を特定する。
4	SLO/alert	SLO/alert — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 観測値をSLOとrunbookへ結び、秘密と高cardinality labelを避けます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

RustNoteのSLI、SLO、alert条件、runbook linkを三つ定義してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
Availability: good responses/eligible >=99.9% 30d; alert burn rate  
Latency: p95 GET /notes <300ms; alert 15m+1h  
Jobs: 99% mail <5m; alert oldest_age>10m
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 観測値をSLOとrunbookへ結び、秘密と高cardinality labelを避けます。
- ✓ tracing・metrics・health・SLOとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

benchmark・profile・allocation

UNIT 08 ・ CONCEPT

benchmark・profile・allocation

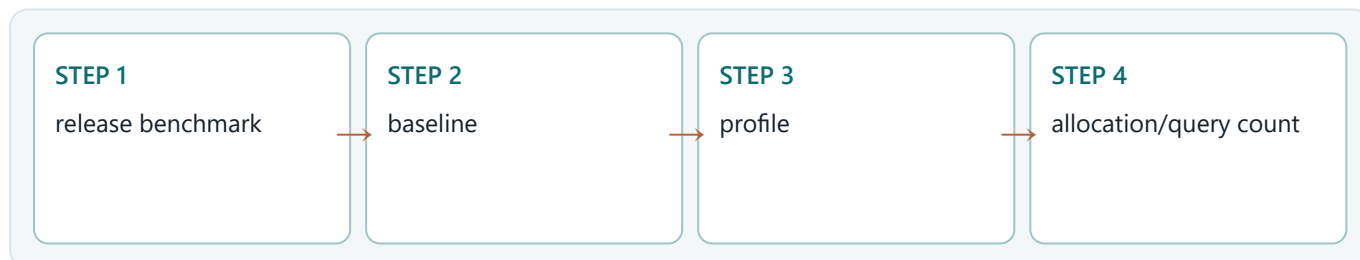
性能改善は計測前に行いません。release buildで代表入力をbenchmarkし、wall time、CPU、allocation、query数を測り、profileでhot spotを特定します。

この単元の到達点

- release benchmark
- baseline
- profile
- allocation/query count

なぜバックエンドで必要か

microbenchmarkだけでなくHTTP+DBのend-to-end latencyを別に測ります。clone削減よりN+1 query解消の方が大きい場合があります。



先に答え

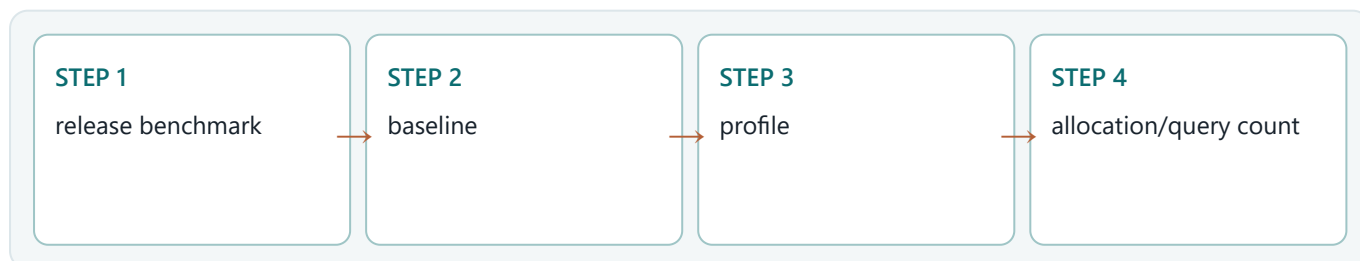
性能はbaseline→profile→仮説→変更→再計測の順で改善します。

確認する問い

benchmark・profile・allocationを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 08 ・ MENTAL MODEL

benchmark・profile・allocationを図でつかむ



性能改善は計測前に行いません。release buildで代表入力をbenchmarkし、wall time、CPU、allocation、query数を測り、profileでhot spotを特定します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認しま

す。

順序	観察対象	初心者が確認すること
1	release benchmark	release benchmark — 入力と前提を確認し、境界を曖昧にしない。
2	baseline	baseline — 値がどの順で変換されるかを追跡する。
3	profile	profile — 失敗を再現し、型またはログから原因を特定する。
4	allocation/query count	allocation/query count — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

benchmark・profile・allocationとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08・MINIMUM CODE

まず動く最小コード

benchmark・profile・allocationだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn list_titles(notes:&[Note])->Vec<String>{notes.iter().filter(|n|!n.done).map(|n|n.title.as_str()).collect()}
// criterion: 10/1000/100000 rowsで測定しbaseline保存
```

●●● 実行コマンド

```
cargo bench
# profileはOS対応toolでrelease binaryを計測
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn list_titles(notes:&[Note])->Vec<&str> {notes.iter().filter(n !n.done).m</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>benchmark・profile・allocation</code>	入力、処理、出力を分離して読みます。
<code>benchmark・profile・allocation</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力release benchmarkの条件を満たすか確認する。
- ✓ baselineを実行し、途中の型と状態を追う。
- ✓ profileで失敗を成功経路から分離する。
- ✓ allocation/query countにより、出力と副作用を検証する。

型を声に出す benchmark・profile・allocationに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

1000 notes: 12.4 μs ± 0.4
allocations: output Vecのみ

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
baseline: owned_titles clones String
candidate: borrowed_titles returns &str
measure throughput/allocations
採用条件: hot pathかつ意味ある改善、API lifetime複雑化を上回る
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08 · BACKEND APPLICATION

バックエンドのどこで使うか

microbenchmarkだけでなくHTTP+DBのend-to-end latencyを別に測ります。clone削減よりN+1 query解消の方が大きい場合があります。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	release benchmarkに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	baselineの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	profileまたはallocation/query countの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

benchmark・profile・allocationを再現して切り分ける

benchmark・profile・allocationの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: release benchmark
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: baseline
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: profile
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: allocation/query count

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	release benchmark	release benchmark — 入力と前提を確認し、境界を曖昧にしない。
2	baseline	baseline — 値がどの順で変換されるかを追跡する。
3	profile	profile — 失敗を再現し、型またはログから原因を特定する。
4	allocation/query count	allocation/query count — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 性能はbaseline→profile→仮説→変更→再計測の順で改善します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

cloneを使う実装と借用する実装をbenchmarkし、可読性と実測差から採否を判断してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
baseline: owned_titles clones String
candidate: borrowed_titles returns &str
measure throughput/allocations
採用条件: hot pathかつ意味ある改善、API lifetime複雑化を上回る
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 性能はbaseline→profile→仮説→変更→再計測の順で改善します。
- ✓ benchmark・profile・allocationとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

load test・backpressure・capacity

UNIT 09・CONCEPT

load test・backpressure・capacity

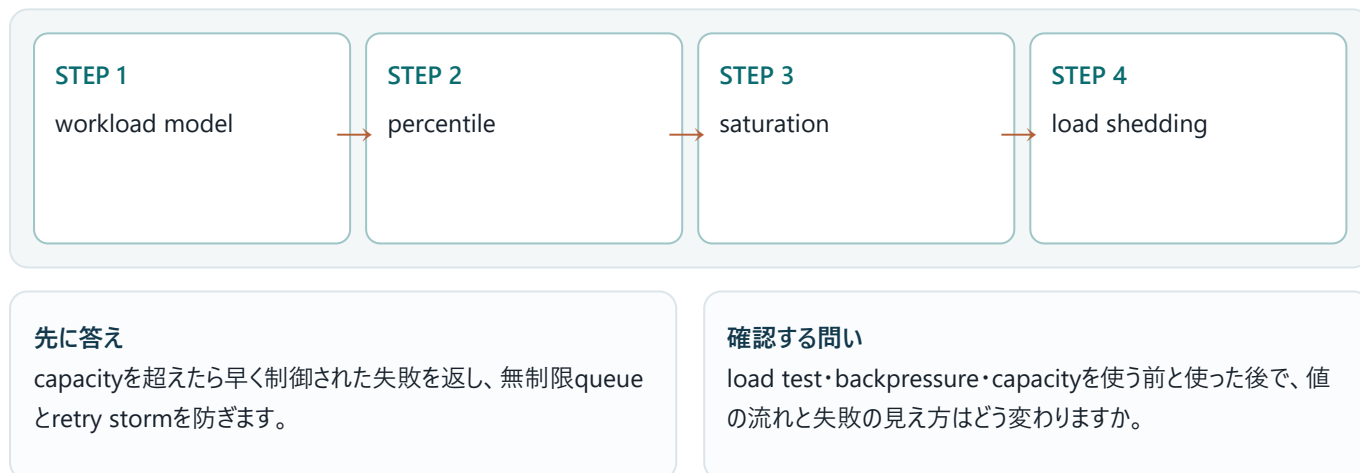
負荷試験は平均だけでなくp95/p99、error、DB latency、queue、CPU、memoryを一定rateで観測します。過負荷で無制限待機せず、Semaphore・queue・timeoutで劣化を制御します。

この単元の到達点

- workload model
- percentile
- saturation
- load shedding

なぜバックエンドで必要か

test dataとTurso costを管理し、本番へ無断負荷を掛けません。rate limitとretryが負荷を増幅しないか確認します。



UNIT 09・MENTAL MODEL

load test・backpressure・capacityを図でつかむ



負荷試験は平均だけでなくp95/p99、error、DB latency、queue、CPU、memoryを一定rateで観測します。過負荷で無制限待機せず、Semaphore・queue・timeoutで劣化を制御します。図では左から右へ処理を追います。各箱の入口では入力 of 型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	workload model	workload model — 入力と前提を確認し、境界を曖昧にしない。
2	percentile	percentile — 値がどの順で変換されるかを追跡する。
3	saturation	saturation — 失敗を再現し、型またはログから原因を特定する。
4	load shedding	load shedding — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

load test・backpressure・capacityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09 · MINIMUM CODE

まず動く最小コード

load test・backpressure・capacityだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
Scenario: 80% GET /notes, 15% POST /notes, 5% login
Ramp: 1→20→100 concurrent
Observe: p95/p99, 4xx/5xx, DB, CPU, RSS, queue
Pass: p95<500ms, 5xx<0.1%, no unbounded growth
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
Scenario: 80% GET /notes, 15% POST /notes, 5% login	この行が受け取る値、作る値、副作用の有無を確認します。
Ramp: 1→20→100 concurrent	この行が受け取る値、作る値、副作用の有無を確認します。
Observe: p95/p99, 4xx/5xx, DB, CPU, RSS, queue	この行が受け取る値、作る値、副作用の有無を確認します。
Pass: p95<500ms, 5xx<0.1%, no unbounded growth	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 workload model の条件を満たすか確認する。
- ✓ percentile を実行し、途中の型と状態を追う。
- ✓ saturation で失敗を成功経路から分離する。
- ✓ load shedding により、出力と副作用を検証する。

型を声に出す load test・backpressure・capacity に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
normal: p95 210ms
overload: 503 busy within 200ms instead of 30s timeout
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 09 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
let permit=timeout(Duration::from_millis(100),db_limit.acquire()).await.map_err(|_|AppError::Busy)?;let result=query().await;drop(permit);
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

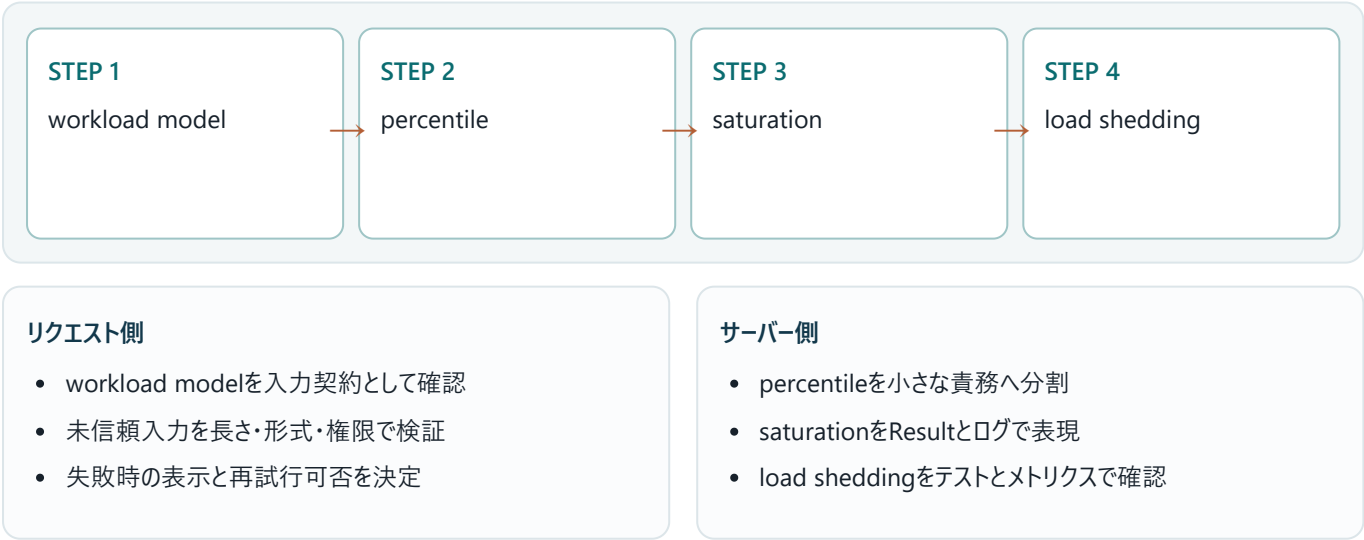
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09・BACKEND APPLICATION

バックエンドのどこで使うか

test dataとTurso costを管理し、本番へ無断負荷を掛けません。rate limitとretryが負荷を増幅しないか確認します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。

✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	workload modelに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	percentileの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	saturationまたはload sheddingの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

load test・backpressure・capacityを再現して切り分ける

load test・backpressure・capacityの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: workload model
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: percentile
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: saturation
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: load shedding

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	workload model	workload model — 入力と前提を確認し、境界を曖昧にしない。
2	percentile	percentile — 値がどの順で変換されるかを追跡する。
3	saturation	saturation — 失敗を再現し、型またはログから原因を特定する。
4	load shedding	load shedding — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 capacityを超えたら早く制御された失敗を返し、無制限queueとretry stormを防ぎます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

DB同時数10のSemaphoreで100 requestを流し、待機timeout後503になるcapacity testを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
let permit=timeout(Duration::from_millis(100),db_limit.acquire()).await.map_err(|_|AppError::Busy)?;let result=query().await;drop(permit);
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ capacityを超えたら早く制御された失敗を返し、無制限queueとretry stormを防ぎます。
- ✓ load test・backpressure・capacityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

CI・dependency・supply chain・incident

UNIT 10・CONCEPT

CI・dependency・supply chain・incident

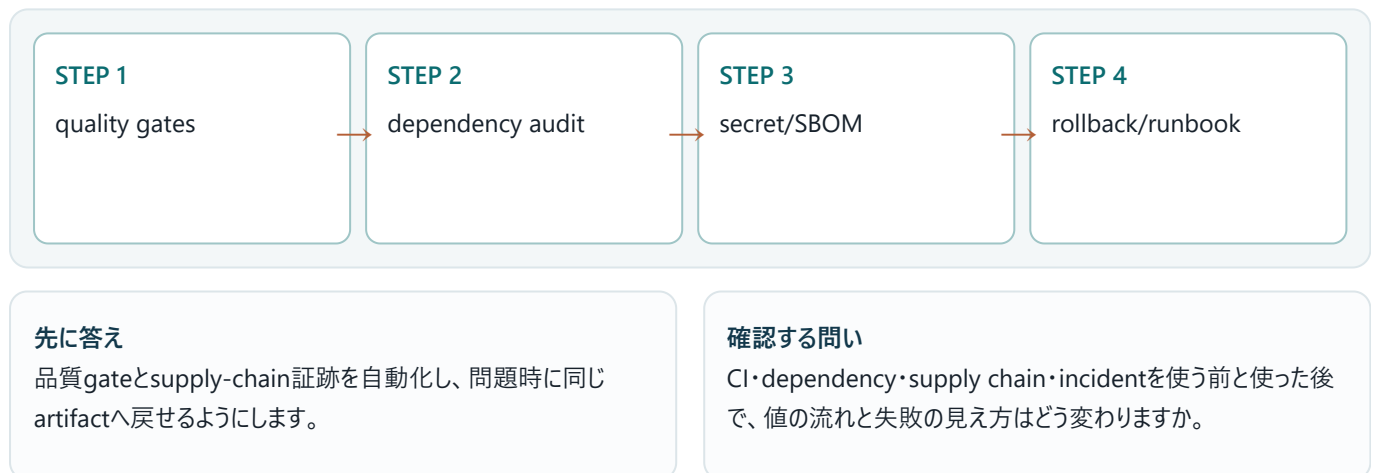
CIはfmt、clippy、test、security test、build、asset bundle、container smokeを順に実行します。依存監査、license、secret scan、SBOM、provenanceを追加し、incident時のrollbackと証跡を準備します。

この単元の到達点

- quality gates
- dependency audit
- secret/SBOM
- rollback/runbook

なぜバックエンドで必要か

Topcoatの破壊的変更を避けるためexact版とlockを守り、更新PRはchangelog、compile、E2E、bundle差を確認します。



UNIT 10・MENTAL MODEL

CI・dependency・supply chain・incidentを図でつかむ



CIはfmt、clippy、test、security test、build、asset bundle、container smokeを順に実行します。依存監査、license、secret scan、SBOM、provenanceを追加し、incident時のrollbackと証跡を準備します。図では左から右へ処理を追います。各箱の入口では入力の方と信頼度、出口では結果の方と副作用を確認します。

順序	観察対象	初心者が確認すること
1	quality gates	quality gates — 入力と前提を確認し、境界を曖昧にしない。
2	dependency audit	dependency audit — 値がどの順で変換されるかを追跡する。
3	secret/SBOM	secret/SBOM — 失敗を再現し、型またはログから原因を特定する。
4	rollback/runbook	rollback/runbook — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

CI・dependency・supply chain・incidentとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

CI・dependency・supply chain・incidentだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
cargo fmt --all -- --check
cargo clippy --all-targets --all-features -- -D warnings
cargo test --all-features
cargo build --locked --release
topcoat asset bundle --release
docker build .
# container /health smoke
```

●●● 実行コマンド

```
cargo test --all-features
docker build -t rustnote:test .
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>cargo fmt --all -- --check</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo clippy --all-targets --all-features -- -D warnings</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo test --all-features</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo build --locked --release</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>topcoat asset bundle --release</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力にquality gatesの条件を満たすか確認する。
- ✓ dependency auditを実行し、途中の型と状態を追う。
- ✓ secret/SBOMで失敗を成功経路から分離する。
- ✓ rollback/runbookにより、出力と副作用を検証する。

型を声に出す CI・dependency・supply chain・incidentに関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

`all gates pass → deploy candidate`
`one gate fails → no deploy`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
CI artifacts: test report/binary hash/container digest/SBOM
Incident: request IDs/logs/metrics/deploy version/timeline
Stop: security/data corruption/SLO burn
Rollback: compatible schema + previous image digest
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

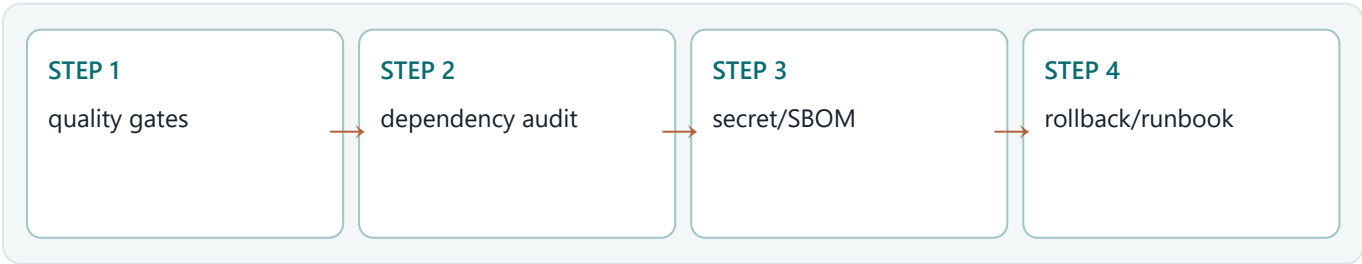
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10 · BACKEND APPLICATION

バックエンドのどこで使うか

Topcoatの破壊的変更を避けるためexact版とlockを守り、更新PRはchangelog、compile、E2E、bundle差を確認します。



リクエスト側

- quality gatesを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- dependency auditを小さな責務へ分割
- secret/SBOMをResultとログで表現
- rollback/runbookをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	quality gatesに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	dependency auditの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	secret/SBOMまたはrollback/runbookの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

CI・dependency・supply chain・incidentを再現して切り分ける

CI・dependency・supply chain・incidentの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: quality gates
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: dependency audit
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: secret/SBOM
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: rollback/runbook

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	quality gates	quality gates — 入力と前提を確認し、境界を曖昧にしない。
2	dependency audit	dependency audit — 値がどの順で変換されるかを追跡する。
3	secret/SBOM	secret/SBOM — 失敗を再現し、型またはログから原因を特定する。
4	rollback/runbook	rollback/runbook — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 品質gateとsupply-chain証跡を自動化し、問題時に同じartifactへ戻せるようにします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

CI失敗と本番incidentのartifact、担当、停止条件、rollback判断を表にしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
CI artifacts: test report/binary hash/container digest/SBOM
Incident: request IDs/logs/metrics/deploy version/timeline
Stop: security/data corruption/SLO burn
Rollback: compatible schema + previous image digest
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 品質gateとsupply-chain証跡を自動化し、問題時に同じartifactへ戻せるようにします。
- ✓ CI・dependency・supply chain・incidentとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

RustNoteのrelease candidateに品質gateを通す

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	unit test・境界値・table-driven	unit testは値の境界と不変条件を短時間で固定します。
02	Repository integration test	Repository testは実schemaでSQL・制約・mapping・transactionを検証します。
03	HTTP handler・status・header test	外部契約はstatus・header・body・cookieを組み合わせて検証します。
04	property test・fuzz・parser	property testは例ではなく成立すべき関係を多数入力で検証します。
05	E2E・browser・accessibility	E2Eは重要journeyとbrowser固有挙動を検証し、失敗artifactを必ず残します。
06	security regression test	脅威をpayloadと不変条件に変換し、修正後も回帰しないtestを残します。
07	tracing・metrics・health・SLO	観測値をSLOとrunbookへ結び、秘密と高cardinality labelを避けます。
08	benchmark・profile・allocation	性能はbaseline→profile→仮説→変更→再計測の順で改善します。
09	load test・backpressure・capacity	capacityを超えたら早く制御された失敗を返し、無制限queueとretry stormを防ぎます。
10	CI・dependency・supply chain・incident	品質gateとsupply-chain証跡を自動化し、問題時に同じartifactへ戻せるようにします。

完了条件

- ✓ domain/DB/HTTP/E2E/security testがある
- ✓ SLO dashboardとalert runbookがある
- ✓ load時に制御された503を返す
- ✓ container digestとSBOMを保存する

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. unit test・境界値・table-drivenを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. Repository integration testを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. HTTP handler・status・header testを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. property test・fuzz・parserを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. E2E・browser・accessibilityを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. security regression testを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. tracing・metrics・health・SLOを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. benchmark・profile・allocationを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. load test・backpressure・capacityを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. CI・dependency・supply chain・incidentを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. unit test・境界値・table-driven

unit test・境界値・table-drivenとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: unit testは値の境界と不変条件を短時間で固定します。

02. Repository integration test

Repository integration testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: Repository testは実schemaでSQL・制約・mapping・transactionを検証します。

03. HTTP handler・status・header test

HTTP handler・status・header testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 外部契約はstatus・header・body・cookieを組み合わせで検証します。

04. property test・fuzz・parser

property test・fuzz・parserとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: property testは例ではなく成立すべき関係を多数入力で検証します。

05. E2E・browser・accessibility

E2E・browser・accessibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: E2Eは重要journeyとbrowser固有挙動を検証し、失敗artifactを必ず残します。

06. security regression test

security regression testとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 脅威をpayloadと不変条件に変換し、修正後も回帰しないtestを残します。

07. tracing・metrics・health・SLO

tracing・metrics・health・SLOとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 観測値をSLOとrunbookへ結び、秘密と高cardinality labelを避けます。

08. benchmark・profile・allocation

benchmark・profile・allocationとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 性能はbaseline→profile→仮説→変更→再計測の順で改善します。

09. load test・backpressure・capacity

load test・backpressure・capacityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: capacityを超えたら早く制御された失敗を返し、無制限queueとretry stormを防ぎます。

10. CI・dependency・supply chain・incident

CI・dependency・supply chain・incidentとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 品質gateとsupply-chain証跡を自動化し、問題時に同じartifactへ戻せるようにします。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
unit test・境界値・table-driven	unit testは値の境界と不変条件を短時間で固定します。	frameworkなしで不変条件を固定すると、web・DB変更後もdomain契約を速く確認できます。
Repository integration test	Repository testは実schemaでSQL・制約・mapping・transactionを検証します。	mockだけではSQL typo、列順、migration差を検出できません。remote Tursoは別の少数contract testで補います。
HTTP handler・status・header test	外部契約はstatus・header・body・cookieを組み合わせて検証します。	HTTP testはdomainとDBを統合し、利用者とproxyが見る外部契約を固定します。internal error文字列がbodyへ漏れないことも確認します。
property test・fuzz・parser	property testは例ではなく成立すべき関係を多数入力で検証します。	path/query/cursor/token parserは攻撃者が任意byteを送れます。fuzz targetでpanic、巨大割当、極端な遅さを探します。
E2E・browser・accessibility	E2Eは重要journeyとbrowser固有挙動を検証し、失敗artifactを必ず残します。	flakyなsleepではなく要素やnetworkの条件を待ち、失敗時にscreenshot、HTML、console、traceを保存します。
security regression test	脅威をpayloadと不変条件に変換し、修正後も回帰しないtestを残します。	security testは『攻撃できないはず』ではなく、具体的payload、期待status、DB不変、log非漏えいをassertします。
tracing・metrics・health・SLO	観測値をSLOとrunbookへ結び、秘密と高cardinality labelを避けます。	route labelはtemplateを使いIDを含めずcardinality爆発を防ぎます。alertはCPUだけでなくerror budgetと利用者症状へ寄せます。
benchmark・profile・allocation	性能はbaseline→profile→仮説→変更→再計測の順で改善します。	microbenchmarkだけでなくHTTP+DBのend-to-end latencyを別に測ります。clone削減よりN+1 query解消の方が大きい場合があります。
load test・backpressure・capacity	capacityを超えたら早く制御された失敗を返し、無制限queueとretry stormを防ぎます。	test dataとTurso costを管理し、本番へ無断負荷を掛けません。rate limitとretryが負荷を増幅しないか確認します。
CI・dependency・supply chain・incident	品質gateとsupply-chain証跡を自動化し、問題時に同じartifactへ戻せるようにします。	Topcoatの破壊的変更を避けるためexact版とlockを守り、更新PRはchangelog、compile、E2E、bundle差を確認します。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。